

NATURAL LANGUAGE PROCESSING

Chapter 5: Word Embeddings

Introduction

- In the previous chapters, we ignored the nearby context of a word.
- Our bag-of-words concept jumbled all the words from each document together into a statistical bag.
- In this chapter, you'll create much smaller bags of words from a "neighborhood" of only a few words, typically fewer than 10 tokens.
- You'll also ensure that these neighborhoods of meaning don't spill over into adjacent sentences.

Introduction

- This process will help focus your word vector training on the relevant words.
- Our new word vectors will be able to identify synonyms, antonyms, or words that just belong to the same category, such as people, animals, places, plants, names, or concepts.

Introduction

- Well, what are these word vectors good for? Say you're trying to recall a famous person's name, but you only have a general impression of them, like maybe this:

She invented something to do with physics in Europe in the early 20th century.

- With word vectors, you can search for words or names that combine the meaning of the words "woman," "Europe," "physics," "scientist," and "famous," and that would get you close to the token "Marie Curie" that you're looking for.

Introduction

- All you have to do to make that happen is add up the word vectors for each of those words that you want to combine:

answer_vector = wv['woman'] + wv['Europe'] + wv[physics'] + wv['scientist']

Introduction

- In this chapter, we'll see the exact way to do this query. And we even show how to subtract gender bias from the word vectors used to compute the answer:

*answer_vector = wv['woman'] + wv['Europe'] + wv[physics'] + wv['scientist'] - wv['male'] - 2 * wv['man']*

Analogy Questions

- What if you could rephrase your question as an analogy question? What if your “query” was something like this:

Who is to nuclear physics what Louis Pasteur is to germs?

- With word vectors, the solution is as simple as subtracting “germs” from “Louis Pasteur” and then adding in some “physics”:

answer_vector = wv['Louis_Pasteur'] - wv['germs'] + wv['physics']

Analogy Questions

- And if you're interested in trickier analogies about people in unrelated fields, such as musicians and scientists, you can do that, too:

Who is the Marie Curie of music?

or

Marie Curie is to science as who is to music?

Then we can just reformatize the equation:

$$wv['Marie_Curie'] - wv['science'] + wv['music']$$

- Word vectors can answer these vague questions and analogy problems, as long as the word vector for the answer exists in your word vector vocabulary.

Word Vectors

- In 2012, Thomas Mikolov, an intern at Microsoft, found a way to encode the meaning of words in a modest number of vector dimensions
- Mikolov trained a neural network to predict word occurrences near each target word.
- In 2013, once at Google, Mikolov and his teammates released the software for creating these word vectors and called it **Word2vec**.



Word Vectors

- Word2vec learns the meaning of words merely by processing a large corpus of unlabeled text.
- **No one has to label** the words in the Word2vec vocabulary; No one has to tell Word2vec that soccer is a sport, or that a team is a group of people.
- Word2vec can learn that and much more, all on its own!
- All you need is a corpus large enough to mention Marie Curie near other words associated with science.
- This unsupervised nature of Word2vec is what makes it so powerful. *The world is full of unlabeled, uncategorized, unstructured natural language text.*

Supervised Vs Unsupervised

Supervised learning

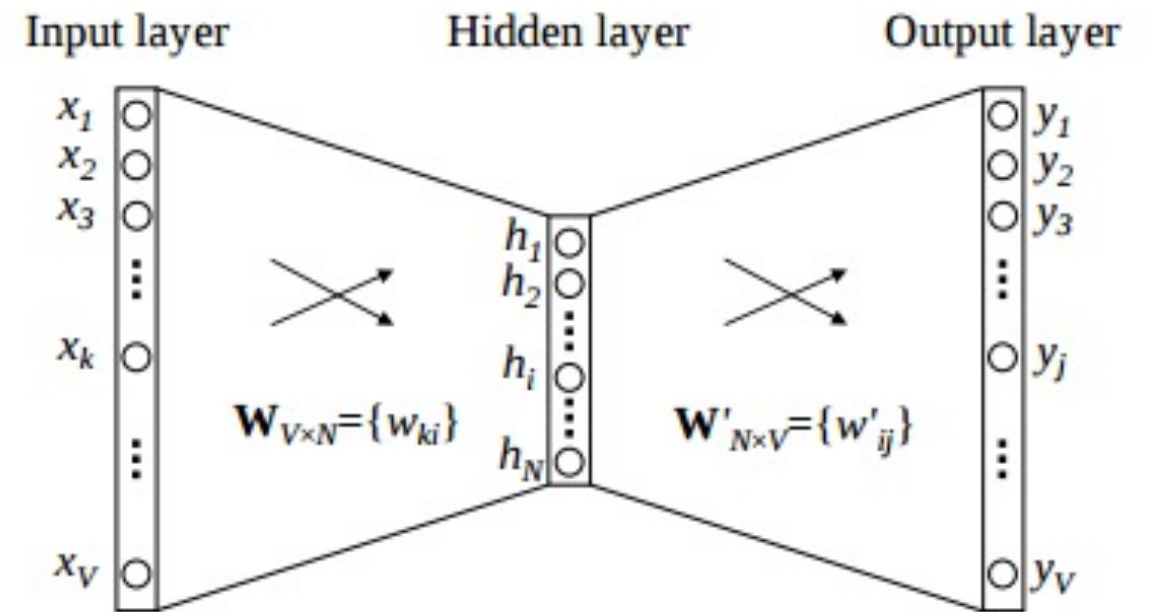
- The training data must be labeled in some way.
- A supervised model can only get better if it can measure the difference between the expected output (the label) and its predictions.

Unsupervised learning

- You train the model to perform a task, but without any labels, only the raw data.
- The model finds patterns in the relationships between the data points themselves.
- An unsupervised model can get smarter (more accurate) just by throwing more data at it.

Word Vectors

- All words in a corpus will be represented by numerical vectors. weights can be added and subtracted to create new word vectors that mean something.
- A mental model that may help you understand word vectors is to think of word vectors as a list of weights or scores. Each weight or score is associated with a specific dimension of meaning for that word.



Vector-Oriented Reasoning

- With Word2Vec's vector-oriented reasoning, you can do math with word vectors and the answer makes sense when you translate the vectors back into words.
- You can add and subtract word vectors to reason about the words they represent and answer questions similar to the previous examples.

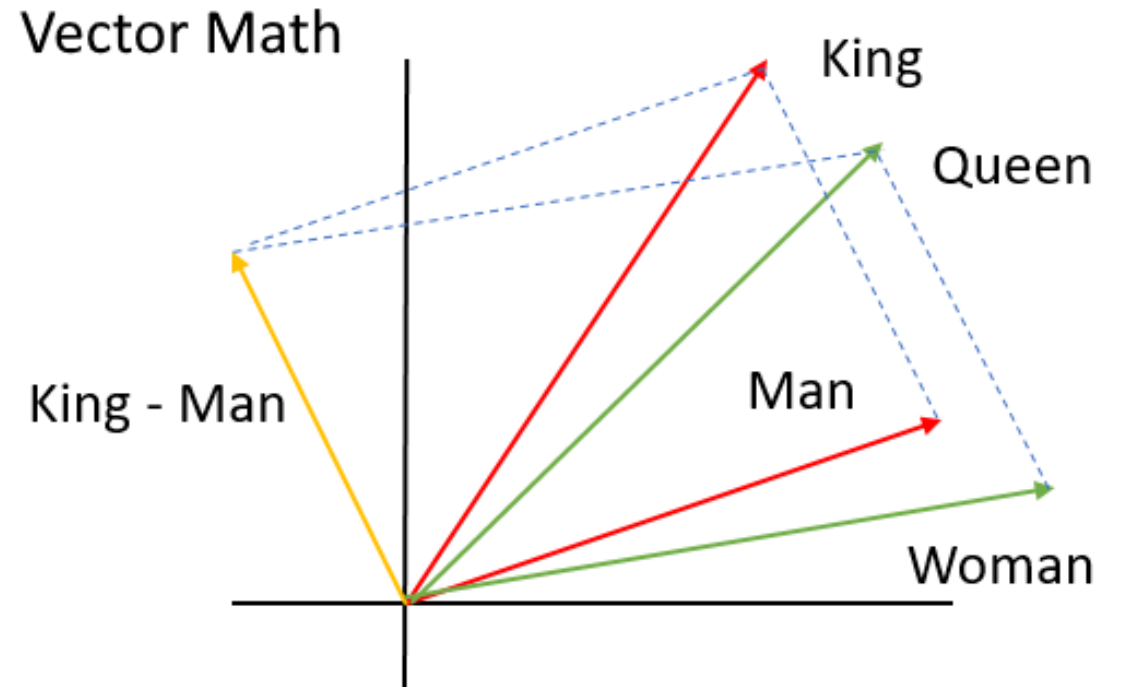
Vector-Oriented Reasoning

- Now, with word vectors, questions like

$$\text{King} + \text{Woman} - \text{Man} = ?$$

can be solved with vector algebra, to give us the answer:

Queen



Vector-Oriented Reasoning

- Now, with word vectors, questions like

$$\textit{King} + \textit{Woman} - \textit{Man} = ?$$

can be solved with vector algebra, to give us the answer:

Queen

$$\begin{bmatrix} 0.0168 \\ 0.007 \\ 0.247 \\ \dots \end{bmatrix} + \begin{bmatrix} 0.093 \\ -0.028 \\ -0.214 \\ \dots \end{bmatrix} - \begin{bmatrix} 0.104 \\ 0.0883 \\ -0.318 \\ \dots \end{bmatrix} = \begin{bmatrix} 0.006 \\ -0.109 \\ 0.352 \\ \dots \end{bmatrix}$$

Vector-Oriented Reasoning

- After adding and subtracting word vectors, your resultant vector will almost never exactly equal one of the vectors in your word vector vocabulary.
- Word2vec word vectors usually have 100s of dimensions, each with continuous real values.
- Nonetheless, the vector in your vocabulary that is closest to the resultant will often be the answer to your NLP question.

Vector-Oriented Reasoning

****Note:**

- The initial paper in 2013 by Mikolov and his colleagues was able to achieve an answer accuracy of only 40%. But back in 2013, the approach outperformed any other semantic reasoning approach by a significant margin (11%).
- Since the initial publication, the performance of Word2vec has improved further. This was accomplished by training it on extremely large corpora.
- The reference implementation was trained on the 100 billion words from the Google News Corpus.

Compute Word2Vec Representations

- Word vectors represent the semantic meaning of words as vectors in the context of the training corpus.
- This allows you to reason about the meaning of words in more general ways with vector algebra. But to calculate these vector representations, there are two possible ways to train Word2vec embeddings:
 - *The skip-gram approach*: predicts the context of words (output words) from a word of interest (the input word).
 - *The continuous bag-of-words (CBOW) approach*: predicts the target word (the output word) from the nearby words (input words).

Compute Word2Vec Representations

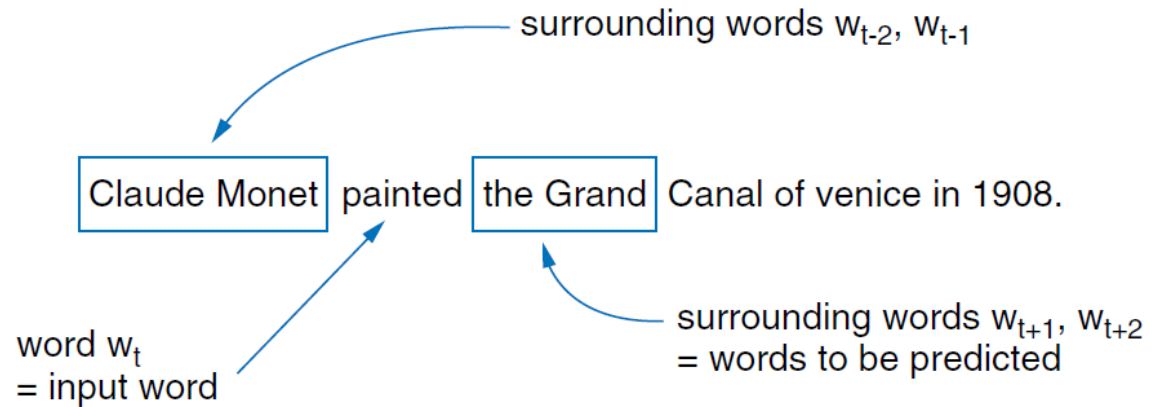
- We'll see how and when to use each of these to train a Word2vec model later.
- The computation of the word vector representations can be resource intensive. Luckily, for most applications, you won't need to compute your own word vectors. You can rely on pretrained representations for a broad range of applications.

Compute Word2Vec Representations

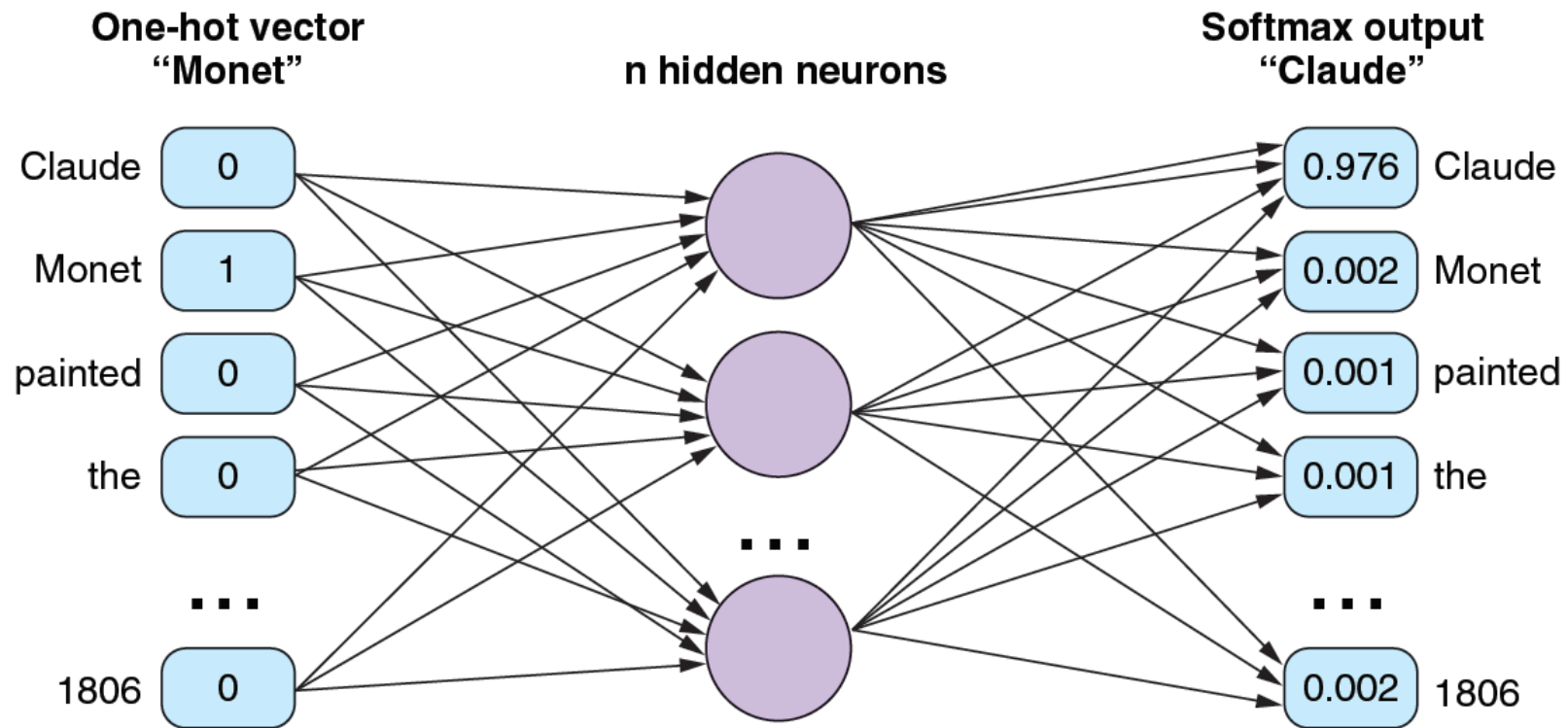
- But if your domain relies on specialized vocabulary or semantic relationships, general purpose word models won't be sufficient.
- For example, if the word "python" should unambiguously represent the programming language instead of the reptile, a domain-specific word model is needed.
- If you need to constrain your word vectors to their usage in a particular domain, you'll need to train them on text from that domain.

Skip-Gram Approach

- In the skip-gram training approach, you're trying to predict the surrounding window of words based on an input word.
- In the sentence "Claude Monet painted the Grand Canal of venice in 1908", "painted" is the training input to the neural network.
- The predicted words for these skipgrams are the neighboring words "Claude," "Monet," "the," and "Grand."



Skip-Gram Approach

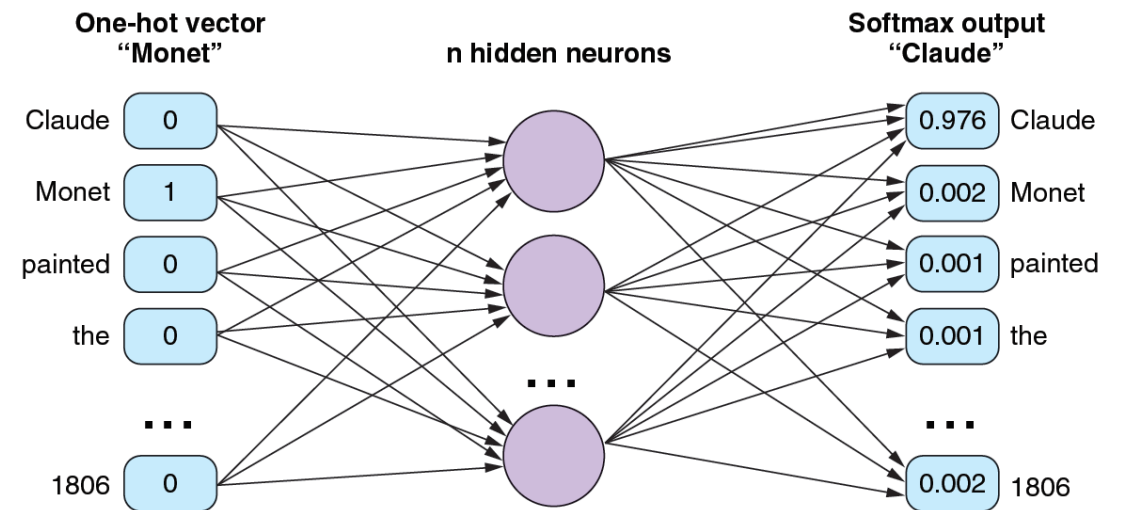


Skip-Gram Approach

- ****What is a skip-gram?** Skip-grams are n-grams that contain gaps because you skip over intervening tokens. In the previous example, you're predicting "Claude" from the input token "painted," and you skip over the token "Monet."

Skip-Gram Approach

- The network consists of two layers of weights, where the hidden layer consists of n neurons; n is the number of vector dimensions used to represent a word.
- Both the input and output layers contain M neurons, where M is the number of words in the model's vocabulary.
- The output layer activation function is a softmax, which is commonly used for classification problems.



Learning Vector Representations

- So if you want to train a Word2vec model using a skip-gram window size (radius) of two words, you're considering the two words before and after each target word.
- You would then use a 5-gram tokenizer to turn a sentence like "Claude Monet painted the Grand Canal of Venice in 1806." into 10 5-grams with the input word at the center, one for each of the 10 words in the original sentence.

Learning Vector Representations

Input word w_t	Expected output w_{t-2}	Expected output w_{t-1}	Expected output w_{t+1}	Expected output w_{t+2}
Claude			Monet	painted
Monet		Claude	painted	the
painted	Claude	Monet	the	Grand
the	Monet	painted	Grand	Canal
Gran	painted	the	Canal	of
Canal	the	Grand	of	Venice
of	Grand	Canal	Venice	in
Venice	Canal	of	in	1908
in	of	Venice	1908	
1908	Venice	in		

Learning Vector Representations

- The training set consisting of the input word and the surrounding (output) words are now the basis for the training of the neural network.
- In the case of four surrounding words, you would use four training iterations, where each output word is being predicted based on the input word.
- Each of the words are represented as **one-hot vectors** before they are presented to the network.
- The output vector for a neural network doing embedding is similar to a one-hot vector as well.

Learning Vector Representations

- The softmax activation of the output layer nodes (one for each token in the vocabulary) calculates the probability of an output word being found as a surrounding word of the input word.
- The output vector of word probabilities can then be converted into a one-hot vector where the word with the highest probability will be converted to 1, and all remaining terms will be set to 0.

Learning Vector Representations

- After training of the neural network is completed, the weights would have been trained to represent the semantic meaning.
- Thanks to the one-hot vector conversion of the tokens, each row in the weight matrix represents each word from the vocabulary for your corpus.
- After the training, semantically similar words will have similar vectors, because they were trained to predict similar surrounding words.

Learning Vector Representations

- After the training is complete, the output layer of the network can be ignored. Only the weights of the inputs to the hidden layer are used as the embeddings.
- Or in other words: the weight matrix is your word embedding.
- The dot product between the one-hot vector representing the input term and the weights then represents the word vector embedding.

Retrieving Word Vectors

- The weights of a hidden layer in a neural network are often represented as a matrix: one column per input neuron, one row per output neuron.
- This allows the weight matrix to be multiplied by the column vector of inputs coming from the previous layer to generate a column vector of outputs going to the next layer.
- So if you multiply (dot product) a one-hot row vector by the trained weight matrix, you'll get a vector that is one weight from each neuron (from each matrix column).
- This also works if you take the weight matrix and multiply it (dot product) by a one-hot column vector for the word you are interested in.

Retrieving Word Vectors

One-hot vector
in vocabulary
of six words

0	1	0	0	0	0
---	---	---	---	---	---

×

Three neuron
weight matrix

.03	.92	.66
.06	.32	.61
.14	.62	.43
.24	.99	.62
.12	.02	.44
.32	.23	.55

=

The dot product calculation

$(0 \cdot .03) + (1 \cdot .06) + (0 \cdot .14) + (0 \cdot .24) + (0 \cdot .12) + (0 \cdot .32)$
$(0 \cdot .92) + (1 \cdot .32) + (0 \cdot .62) + (0 \cdot .99) + (0 \cdot .02) + (0 \cdot .23)$
$(0 \cdot .66) + (1 \cdot .61) + (0 \cdot .43) + (0 \cdot .62) + (0 \cdot .44) + (0 \cdot .55)$

=

.06
.32
.61

Resulting 3-D word vector

Conversion of one-hot vector to word vector

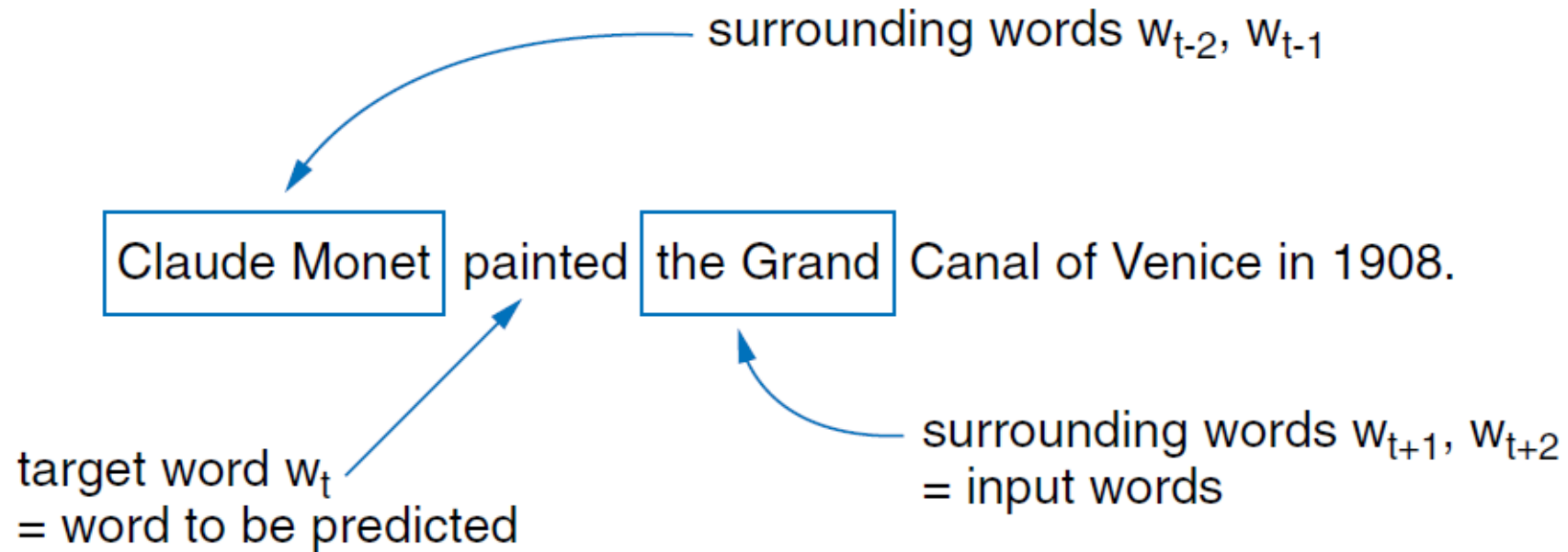
Retrieving Word Vectors

- Of course, the one-hot vector dot product just selects that row from your weight matrix that contains the weights for that word, which is your word vector.
- So you could easily retrieve that row by just selecting it, using the word's row number or index number from your vocabulary.

Continuous Bag-of-Words Approach

- In the continuous bag-of-words approach, you're trying to predict the center word based on the surrounding words.
- Instead of creating pairs of input and output tokens, you'll create a multi-hot vector of all surrounding terms as an input vector.
- The multi-hot input vector is the sum of all one-hot vectors of the surrounding tokens to the center, target token.

Continuous Bag-of-Words Approach



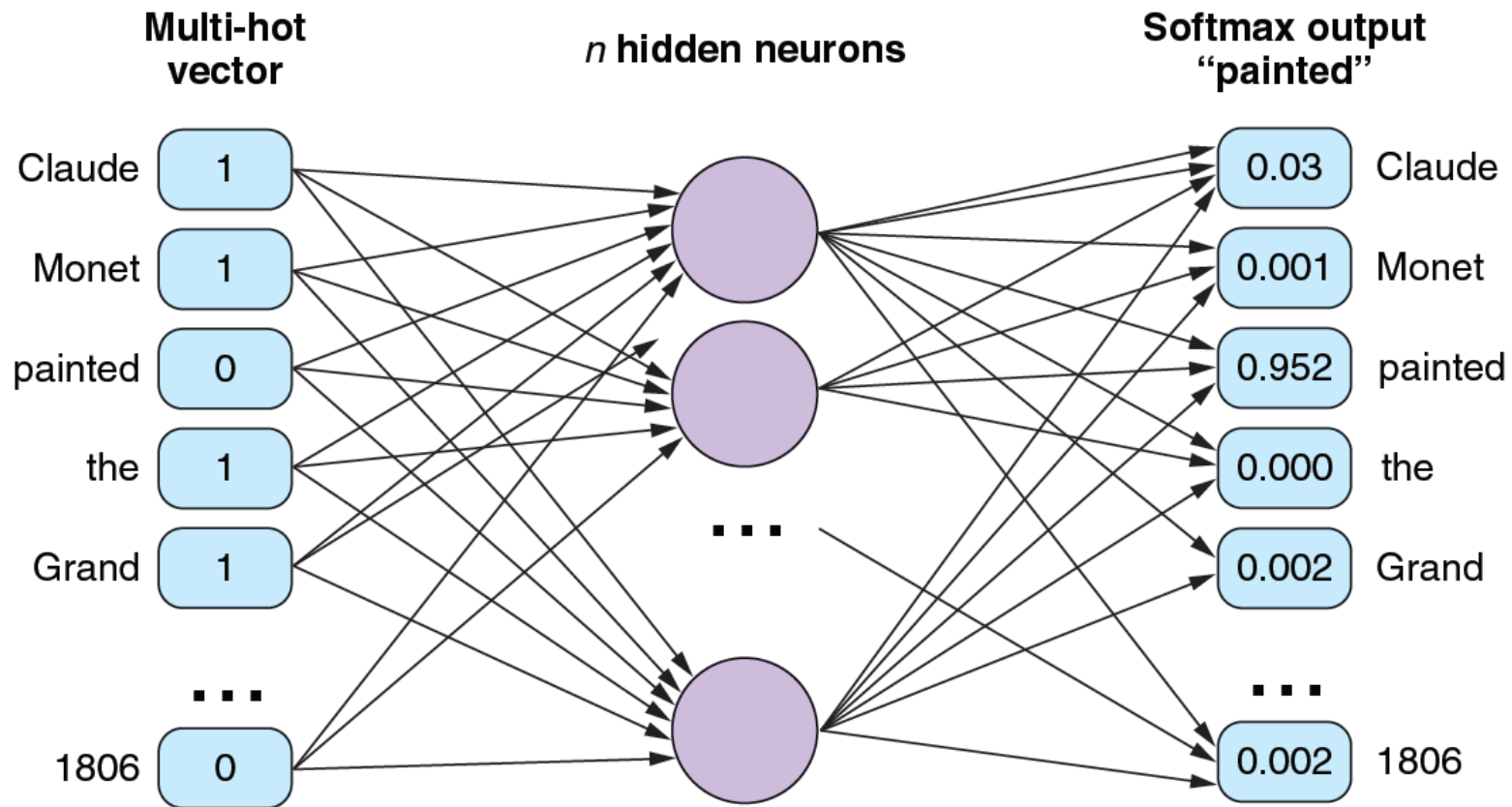
Continuous Bag-of-Words Approach

Input word wt-2	Input word wt-1	Input word wt+1	Input word wt+2	Expected output wt
		Monet	painted	Claude
	Claude	painted	the	Monet
Claude	Monet	the	Grand	painted
Monet	painted	Grand	Canal	the
painted	the	Canal	of	Gran
the	Grand	of	Venice	Canal
Gran	Canal	Venice	in	of
Canal	of	in	1908	Venice
of	Venice	1908		in
Venice	in			1908

Continuous Bag-of-Words Approach

- Based on the training sets, you can create your multi-hot vectors as inputs and map them to the target word as output.
- The multi-hot vector is the sum of the one-hot vectors of the surrounding words' training pairs $w_{t-2} + w_{t-1} + w_{t+1} + w_{t+2}$.
- You then build the training pairs with the multi-hot vector as the input and the target word w_t as the output.
- During the training, the output is derived from the softmax of the output node with the highest probability.

Continuous Bag-of-Words Approach



CBOW vs. BOW

- Previously, we introduced the concept of a bag of words, but how is it different than a continuous bag of words?
- To establish the relationships between words in a sentence you slide a rolling window across the sentence to select the surrounding words for the target word.
- All words within the sliding window are considered to be the content of the continuous bag of words for the target word at the middle of that window.

Continuous Bag of Words

Claude Monet painted the Grand Canal of Venice in 1908.

Claude Monet painted the Grand Canal of Venice in 1908.

Claude Monet painted the Grand Canal of Venice in 1908.

Skip-Grams vs. CBOW: When to Use Which?

- Mikolov highlighted that the skip-gram approach works well with small corpora and rare terms.
- With the skip-gram approach, you'll have more examples due to the network structure.
- But the continuous bag-of-words approach shows higher accuracies for frequent words and is much faster to train.

Subsampling Frequent Tokens

- An accuracy improvement to the original algorithm was to subsample frequent words.
- Common words like “the” or “a” (stop words) often don’t carry significant information.
- **However**, All words carry meaning, including stop words. So stop words shouldn’t be completely ignored or skipped while training your word vectors or composing your vocabulary.

Subsampling Frequent Tokens

- To reduce the emphasis on frequent words like stop words, words are sampled during training in inverse proportion to their frequency.
- The effect of this is similar to the IDF effect on TF-IDF vectors.
- Frequent words are given less influence over the vector than the rarer words.
- Tomas Mikolov used the following equation to determine the probability of sampling a given word.

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

Subsampling Frequent Tokens

- This probability determines whether or not a particular word is included in a particular skip-gram during training.

$$P(wi) = 1 - \sqrt{\frac{t}{f(wi)}}$$

- $f(wi)$: the frequency of the word wi across the corpus
- t : represents a frequency threshold above which you want to apply the subsampling probability.
- The threshold depends on your corpus size, average document length, and the variety of words used in those documents.

Note: Subsampling is pre-implemented in Word2Vec implementations.

Using the gensim.word2vec Module

- Various companies provide their pretrained word vector models. For word vectors, we'll use the gensim library.
- There are also pre-trained models of word2vec ready for download in multiple languages. (<https://code.google.com/archive/p/word2vec/>)
- Then we can load it into a model:

```
from gensim.models.keyedvectors import KeyedVectors
word_vectors = KeyedVectors.load_word2vec_format('/path/to/GoogleNews-
vectors-negative300.bin.gz', binary=True)
```

Using the gensim.word2vec Module

- Working with word vectors can be memory intensive. If your available memory is limited, or if you don't want to wait minutes for the word vector model to load, you can reduce the number of words loaded into memory by passing in the limit keyword argument.
- Here, we load the 200k most common words from the Google News corpus.

```
word_vectors = KeyedVectors.load_word2vec_format('/path/to/GoogleNews-vectors-negative300.bin.gz', binary=True, limit=200000)
```

Using the `gensim.word2vec` Module

- The `gensim.KeyedVectors.most_similar()` method provides an efficient way to find the nearest neighbors for any given word vector.
- The keyword argument `positive` takes a list of the vectors to be added together. Similarly, you can use the negative argument for subtraction and to exclude unrelated terms.
- The argument `topn` determines how many related terms should be provided as a return value.

Using the gensim.word2vec Module

```
word_vectors.most_similar(positive=['cooking', 'potatoes'], topn=5)
[('cook', 0.6973530650138855),
 ('oven_roasting', 0.6754530668258667),
 ('Slow_cooker', 0.6742032170295715),
 ('sweet_potatoes', 0.6600279808044434),
 ('stir_fry_vegetables', 0.6548759341239929)]
```

```
word_vectors.most_similar(positive=['germany', 'france'], topn=1)
[('europe', 0.7222039699554443)]
```

Using the gensim.word2vec Module

- Word vector models also allow you to determine unrelated terms. The gensim library provides a method called *doesn't_match*.
- This determines the most unrelated term of the list, the method returns the term with the highest distance to all other list terms.

```
word_vectors.doesnt_match("potatoes milk cake computer".split())  
'computer'
```

Using the gensim.word2vec Module

- If you want to perform calculations (such as the famous example king + woman - man = queen), you can do that by adding a negative argument to the *most_similar* method call:

```
word_vectors.most_similar(positive=['king', 'woman'], negative=['man'], topn=2)

[('queen', 0.7118192315101624), ('monarch', 0.6189674139022827)]
```

Using the gensim.word2vec Module

- The gensim library also allows you to calculate the similarity between two terms.

```
word_vectors.similarity('princess', 'queen')  
0.70705315983704509
```

Using the gensim.word2vec Module

- If you want to develop your own functions and work with the raw word vectors, you can access them through the square bracket syntax (`[]`) or the `get()` method on a *KeyedVector* instance.
- You can treat the loaded model object as a dictionary where your word of interest is the dictionary key.
- Each float in the returned array represents one of the vector dimensions. In the case of Google's word model, your numpy arrays will have a shape of 1 x 300

```
word_vectors['phone']  
array([-0.01446533, -0.12792969, -0.11572266, -0.22167969, -0.07373047,  
-0.05981445, -0.10009766, -0.06884766, 0.14941406, 0.10107422,  
-0.03076172, -0.03271484, -0.03125 , -0.10791016, 0.12158203,  
0.16015625, 0.19335938, 0.0065918 , -0.15429688, 0.03710938,  
...])
```

Generating Your Own Word Vector Representation

- In some cases, you may want to create your own domain-specific word vector models.
- Doing so can improve the accuracy of your model if your NLP pipeline is processing documents that use words in a way that you wouldn't find on Google News.
- Keep in mind, you need a lot of documents to achieve good results. But in domain-specific cases such as medical texts or transcripts, for example, a domain-specific word model may improve your model accuracy.

Generating Your Own Word Vector Representation

Preprocessing

- First you need to break your documents into sentences and the sentences into tokens. The gensimword2vec model expects a list of sentences, where each sentence is broken up into tokens.
- The training input should look similar to the following structure

```
[  
  ['to', 'provide', 'early', 'intervention/early', 'childhood', 'special',  
   'education', '...'],  
  ['essential', 'job', 'functions'],  
  ['participate', 'as', 'a', 'transdisciplinary', '...']  
  ...  
]
```

Generating Your Own Word Vector Representation

Training

First, set the model parameters:

```
from gensim.models.word2vec import Word2Vec
num_features = 300 # Number of vector dimensions
min_word_count = 3 # Min number of word count to be considered in the
Word2vec model
num_workers = 2 # Number of CPU cores used for the training.
window_size = 6 # Context window size
subsampling = 1e-3 # Subsampling rate for frequent terms
```

Generating Your Own Word Vector Representation

Training

Now to start the training:

```
model = Word2Vec(  
    token_list,  
    workers=num_workers,  
    size=num_features,  
    min_count=min_word_count,  
    window=window_size,  
    sample=subsampling)
```

Depending on your corpus size and your CPU performance, the training will take a significant amount of time. For smaller corpora, the training can be completed in minutes.

Generating Your Own Word Vector Representation

Save the trained model with the following command for later use:

```
model_name = "my_domain_specific_word2vec_model"  
model.save(model_name)
```

And you can load a model that you've trained the same way the google trained model was loaded:

```
from gensim.models.word2vec import Word2Vec  
model_name = "my_domain_specific_word2vec_model"  
model = Word2Vec.load(model_name)
```

GloVe (Global Vectors)

- Word2vec was a breakthrough, but it relies on a neural network model that must be trained using backpropagation which is usually less efficient than direct optimization of a cost function using gradient descent.
- Stanford NLP researchers led by Jeffrey Pennington set about to understand the reason why Word2vec worked so well and to find the cost function that was being optimized.

GloVe (Global Vectors)

- They produced GloVe, an optimized version of the Word2Vec Model.
- GloVe can produce matrices equivalent to the input weight matrix and output weight matrix of Word2vec, producing a language model with the same accuracy as Word2vec but in much less time.
- GloVe speeds the process by using the text data more efficiently.

GloVe (Global Vectors)

- Even though Word2vec first popularized the concept of semantic reasoning with word vectors, your workhorse should probably be GloVe to train new word vector models.
- With GloVe, you'll be more likely to find the global optimum for those vector representations, giving you more accurate results.

GloVe (Global Vectors)

Advantages of GloVe are

- Faster training
- Better RAM/CPU efficiency (can handle larger documents)
- More efficient use of data (helps with smaller corpora)
- More accurate for the same amount of training

FastText

- Researchers from Facebook took the concept of Word2vec one step further by adding a new twist to the model training.
- The new algorithm, which they named fastText, predicts the surrounding n-character grams rather than just the surrounding words, like Word2vec does.
- For example, the word “whisper” would generate the following 2- and 3-character grams:

wh, whi, hi, his, is, isp, sp, spe, pe, per, er

FastText

- fastText trains a vector representation for every n-character gram, which includes words, misspelled words, partial words, and even single characters.
- The advantage of this approach is that it handles rare words much better than the original Word2vec approach.
- As part of the fastText release, Facebook published pretrained fastText models for 294 languages.
- The pretrained models are available at the official fastText website:
<https://fasttext.cc/docs/en/english-vectors.html>

FastText

- The pretrained fastText models provided by Facebook have only been trained on the available Wikipedia corpora.
- Therefore, the vocabulary and accuracy of the models will vary across languages.

```
from gensim.models.fasttext import FastText
ft_model = FastText.load_fasttext_format(model_file=MODEL_PATH)
ft_model.most_similar('soccer')
```

Visualizing Word Relationships

- The semantic word relationships can be powerful and their visualizations can lead to interesting discoveries.
- Here, we demonstrate steps to visualize the word vectors in 2D.
- To get started, load all the word vectors from the Google Word2vec model.

```
from gensim.models.keyedvectors import KeyedVectors
wv = KeyedVectors.load_word2vec_format('/path/to/GoogleNews-vectors-
negative300.bin.gz', binary=True)
len(wv.vocab)
3000000
```

Visualizing Word Relationships

- **WARNING** The Google News Word2vec model is huge: three million words with 300 vector dimensions each. The complete word vector model requires 3 GB of available memory.

Visualizing Word Relationships

- This *KeyedVectors* object in gensim now holds a table of three million Word2vec vectors.
- Let's show just 6 words in the vocabulary, starting at the one millionth word.

```
import pandas as pd
vocab = pd.Series(wv.vocab)
vocab.iloc[1000000:1000006]

Illington_Fund Vocab(count:447860, index:2552140)
Illingworth Vocab(count:2905166, index:94834)
Illingworth_Halifax Vocab(count:1984281, index:1015719)
Illini Vocab(count:2984391, index:15609)
IlliniBoard.com Vocab(count:1481047, index:1518953)
Illini_Bluffs Vocab(count:2636947, index:363053)
```

Visualizing Word Relationships

- Notice that compound words and common n-grams are joined together with an underscore character ("_").

```
import pandas as pd
vocab = pd.Series(wv.vocab)
vocab.iloc[1000000:1000006]

Illington_Fund Vocab(count:447860, index:2552140)
Illingworth Vocab(count:2905166, index:94834)
Illingworth_Halifax Vocab(count:1984281, index:1015719)
Illini Vocab(count:2984391, index:15609)
IlliniBoard.com Vocab(count:1481047, index:1518953)
Illini_Bluffs Vocab(count:2636947, index:363053)
```

Visualizing Word Relationships

- Now let's retrieve all the Word2vec vectors for US cities so you can use their distances to plot them on a 2D map of meaning.
- Let's load another dataset containing a list of cities and states (regions) from around the world. For that, we can use the python library nlpia:

```
from nlpia.data.loaders import get_data
cities = get_data('cities')
cities.head(1).T
```

```
geonameid 3039154
name El Tarter
asciiname El Tarter
alternatenames Ehl Tarter,?? ??????
latitude 42.5795
longitude 1.65362
feature_class P
feature_code PPL
...
```

Visualizing Word Relationships

- This dataset from Geocities contains a lot of information, including latitude, longitude, and population.
- From which, we'll only use the city names.

```
from nlpia.data.loaders import get_data
cities = get_data('cities')
cities.head(1).T
```

```
geonameid 3039154
name El Tarter
asciiname El Tarter
alternatenames Ehl Tarter,?? ??????
latitude 42.5795
longitude 1.65362
feature_class P
feature_code PPL
...
```

Let's focus on just the United States for now:

Visualizing Word Relationships

```
us = cities[(cities.country_code == 'US')
            & (cities.admin1_code.notnull())].copy()
states = pd.read_csv('http://www.fonz.net/blog/wp-
content/uploads/2008/04/states.csv')
states = dict(zip(states.Abbreviation, states.State))
us['city'] = us.name.copy()
us['st'] = us.admin1_code.copy()
us['state'] = us.st.map(states)
us[us.columns[-3:]].head()
```

	city	st	state
geonameid			
4046255	Bay Minette	AL	Alabama
4046274	Edna	TX	Texas
4046319	Bayou	La	Batre AL
Alabama			
4046332	Henderson	TX	Texas
4046430	Natalia	TX	Texas

Visualizing Word Relationships

- Now you have a full state name for each city in addition to its abbreviation.
- Let's check to see which of those state names and city names exist in your Word2vec vocabulary.

```
vocab = pd.np.concatenate([us.city, us.st, us.state])  
vocab = np.array([word for word in vocab if word in wv.wv])  
vocab[:5]
```

```
array(['Edna', 'Henderson', 'Natalia', 'Yorktown', 'Brighton'])
```

Visualizing Word Relationships

- There will be a lot of large cities with the same name, like Portland-Oregon and Portland-Maine.
- So let's incorporate into your city vector the state where that city is located.
- To combine the meanings of words in Word2vec, you add the vectors together.

Visualizing Word Relationships

```
city_plus_state = []
for c, state, st in zip(us.city, us.state, us.st):
    if c not in vocab:
        continue
    row = []
    if state in vocab:
        row.extend(wv[c] + wv[state])
    else:
        row.extend(wv[c] + wv[st])
    city_plus_state.append(row)
us_300D = pd.DataFrame(city_plus_state)
```

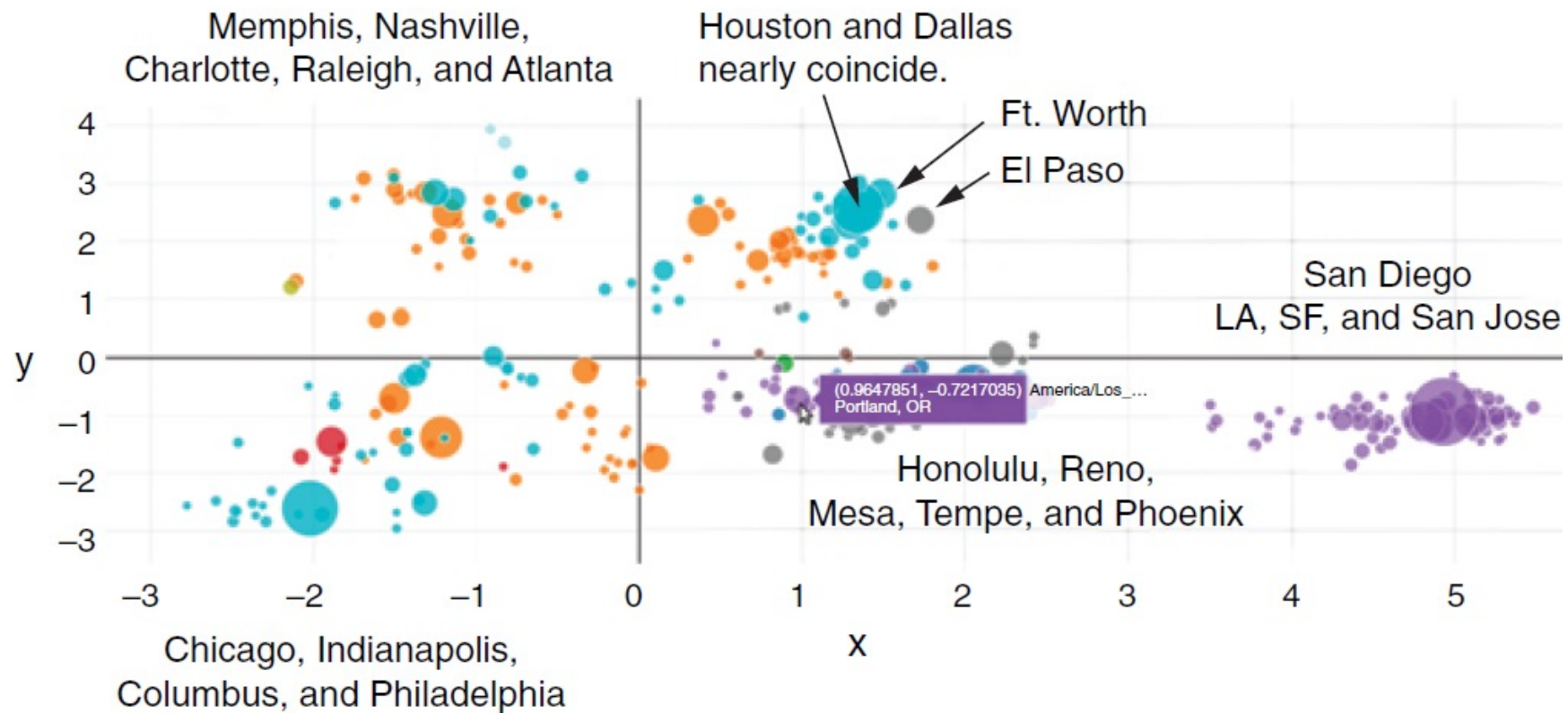
Visualizing Word Relationships

- In the corpus that we're using, cities that are similar in size and culture are clustered close together despite being far apart geographically.
- We use PCA, a dimension reduction algorithm, to reduce the dimensionality of the data into a visualizable one.
- The PCA algorithm ensures that this projection is the best possible view of your data, keeping the vectors as far apart as possible.

Visualizing Word Relationships

```
from matplotlib import pyplot as plt
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
us_300D = get_data('cities_us_wordvectors')
us_2D = pca.fit_transform(us_300D.iloc[:, :300])
plt.scatter(us_2D[0], us_2D[1])
plt.show()
```

Visualizing Word Relationships



Visualizing Word Relationships

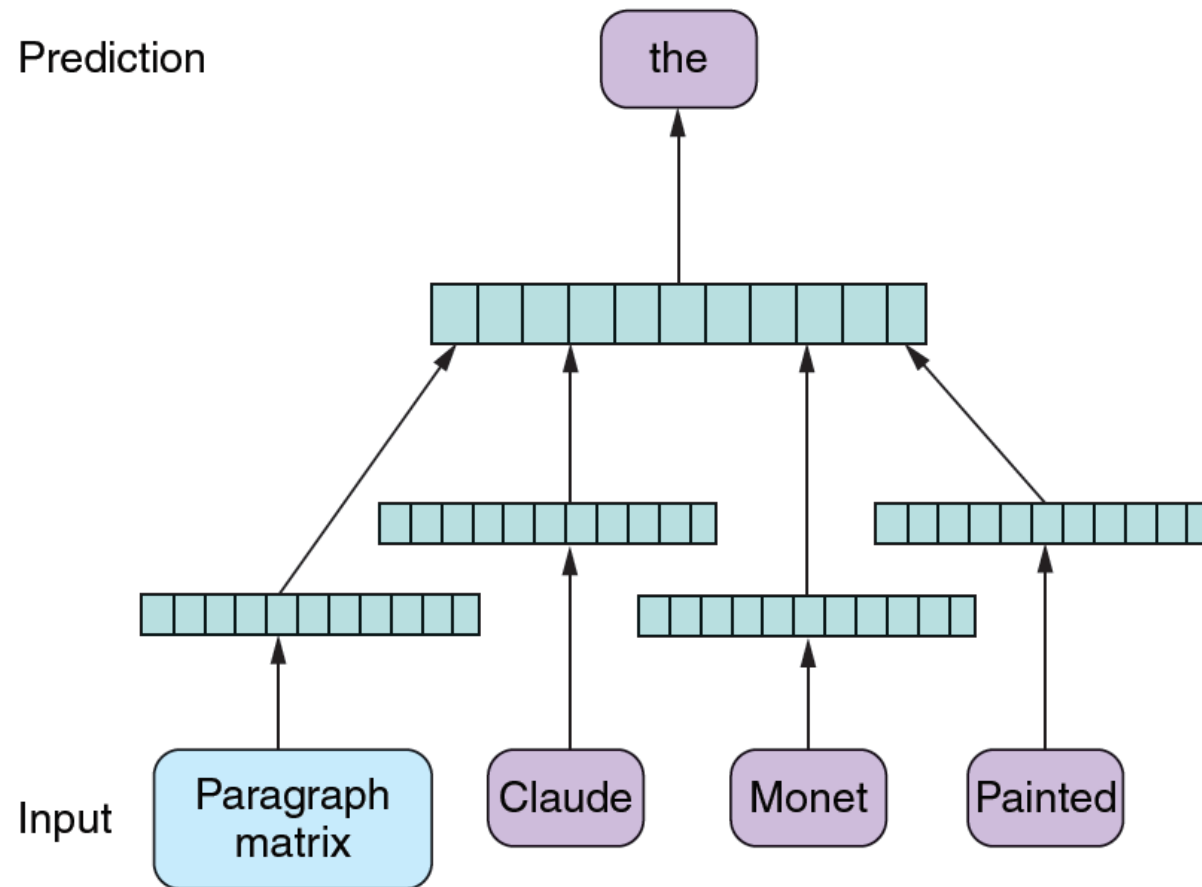
NOTE

- Low semantic distance (distance values close to zero) between word vectors represents high similarity between words. The semantic distance, or “meaning” distance, is determined by the words occurring nearby in the **documents used for training**. The *Word2vec* vectors for two terms are close to each other in word vector space if they are often used in similar contexts.
- **Word vectors are biased!** Word vectors learn word relationships based on the training corpus. Identifying and compensating for biases is a challenge for any NLP practitioner that trains their models on documents written in a biased world.

Document similarity with Doc2vec

- The concept of Word2vec can also be extended to sentences, paragraphs, or entire documents.
- In these cases, the prediction not only considers the previous words, but also the vector representing the paragraph or the document.
- By expanding the concept of Word2vec with an additional document or paragraph vector used for the word prediction, you can now use the trained document vector for various tasks, such as finding similar documents in a corpus

Document similarity with Doc2vec



How to Train Document Vectors

- Similar to the training of word vectors, we use the gensim package to train document vectors.

```
import multiprocessing
num_cores = multiprocessing.cpu_count()
from gensim.models.doc2vec import TaggedDocument, Doc2Vec
from gensim.utils import simple_preprocess
corpus = ['This is the first document ...', 'another document ...']
training_corpus = []
for i, text in enumerate(corpus):
    tagged_doc = TaggedDocument(simple_preprocess(text), [i])
    training_corpus.append(tagged_doc)
model = Doc2Vec(vector_size=100, min_count=2, workers=num_cores, iter=10)
model.build_vocab(training_corpus)
model.train(training_corpus, total_examples=model.corpus_count,
epochs=model.iter)
```

How to Train Document Vectors

- Once the Doc2vec model is trained, you can infer document vectors for new, unseen documents by calling *infer_vector* on the instantiated and trained model:

```
model.infer_vector(simple_preprocess('This is a completely unseen  
document'), steps=10)
```

How to Train Document Vectors

- Doc2Vec can be applied on any real-world task on text data, like document similarity:
You could generate a vector for every document in your corpus and then calculate the cosine distance between each document vector.
- Another common task is to cluster the document vectors of a corpus with something like k-means to create a document classifier.
- Unfortunately, there isn't any major pre-trained models of Doc2Vec. So you have to train your own model if you want to use it.